adatb vizsga

min. 35 pont (30 beugróból és 5 kifejtősből)

TODO: normálformák (BCNF és 4. formálforma), vetítések sémáról sémára, angol

előadás 01
előadás 02
    SQL módosító utasítások
    megszorítások
előadás 04
    nézettáblák
    jogosultságok
előadás 05
    relációs adatmodell-tervezés
előadás 06
    BCNF
    többértékű függőségek
előadás 07
előadás 08
előadás 09
előadás 10
előadás 11

előadás 01

- **adatbázis:** olyan adatok összessége amit egy DBMS (DataBase Management System) kezel
- általános DBMS tud
    - új adatbázist létrehozni, benne *sémákat* definiálni (séma = db logikai szerkezete)
    - lekérdezés, módosítás, DML (Data Manipulation Language)
    - biztonság, hosszú ideig tárolás, etc
    - egyszerre több user hozzáférése

- **data model** részei
  - adatstruktúra
  - adatokon végezhető műveletek
  - megszorítások
- legfontosabb adatmodellek: relációs és féligstrukturált (XML)
- **relációs adatmodell**
  - adatokat kétdimenziós tömbben (tables) & relációkban tároljuk
  - minden attribútumhoz tartozik egy értékkészlet
  - egy reláció soroknak egy halmaza → sorrend nem számít, egy elem csak egyszer szerepelhet
- reláció sorainak halmazát *előfordulásnak* (relation instance) nevezzük
- relációk: $\rho \subset X_1 \times \ldots \times X_n$
  - ahol X1, ... Xn értékkészletei az egyes attribútumoknak
- egy sor elemei: *mezők*
- **megszorítás:** kulcsokkal (aláhúzással jelöljük)
- **relációs algebra:** atomi operandusok & műveletek
  - minden művelet eredménye reláció

TODO: relációs algebra, 02_Relaciok.pdf, 20. oldaltól, 03 fájl rel. alg. optimalizációja

előadás 02

- EXISTS(), ANY() (legalább egyik sorára igaz), ALL()
- <> nem egyenlő
- UNION, INTERSECT, MINUS/EXCEPT
- NATURAL JOIN, CROSS JOIN (Descartes szorzat)
- ... JOIN ... ON ...
- belső összekapcsolás: natural & inner join
  - théta összekapcsolás

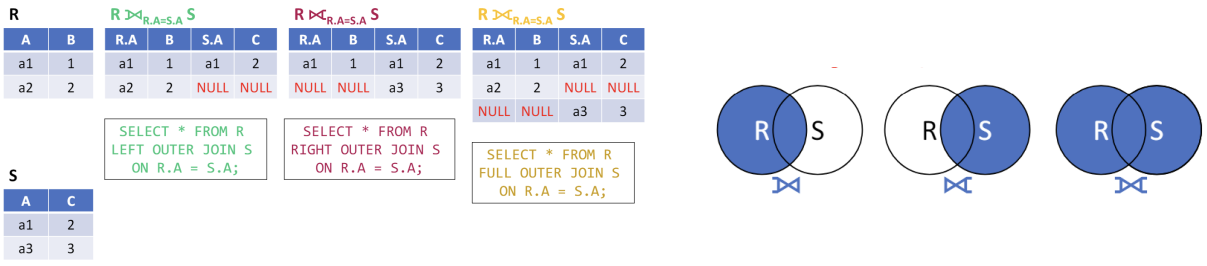
- félig összekapcsolás: ⋉ semi join
  - R(A1, ..., An), S(B1, ..., Bm) sémájú táblák esetén: R ⋉ S ekvivalense: $\Pi_{A1,A2,\ldots,An}(R|X|S))$

```
SELECT R.* FROM R WHERE EXISTS(
    SELECT * FROM S WHERE R.A = S.A
);
```

  - anti join: R ▷S ekvivalense: R – R ⋉ S

```
SELECT R.* FROM R WHERE NOT EXISTS(
    SELECT * FROM S WHERE R.A = S.A
);
```

- külső: outer join
  - van natural outer join (pl. nem írjuk ki hogy R.A = S.A)
  - R sorait aminek nincsenek párjai S-ben **lógó soroknak** nevezzük
    - külső összekapcsolás megőrzi ezeket NULL értékkel helyettesítve a hiányzó értékeket



- aggregációk: AVG, SUM, ...
  - NULL sosem számít kiértékeléskor!

SQL módosító utasítások

- DML = Data Manipulation Language

```
INSERT INTO <reláció> VALUES ( <attribútum lista> );
DELETE FROM <reláció> WHERE <feltétel>;
```

```
UPDATE <reláció> SET <attribútum értékadások listája> WHERE
<sorokra vonatkozó feltétel>;
```

```
CREATE TABLE Vendégek (
    név CHAR(30) PRIMARY KEY,
    cím CHAR(50) DEFAULT ’123 Sesame St.’,
);
```

- DDL: Data Definition Language, adatobjektumokat, megszorításokat etc. hozhatunk vele létre

```
DROP TABLE <név>;
```

- adattípusok SQL-ben
  - INT(EGER), REAL/FLOAT, CHAR(n) (rögzített hosszú), VARCHAR(n) (változó hosszú)
  - DATE: YYYY-MM-DD (pl. DATE ’2007-09-30’)
  - TIME: TIME ’hh:mm:ss’
- egyattribútumos kulcs: UNIQUE vagy PRIMARY KEY kulcsszóval

```
CREATE TABLE Teák (
    név CHAR(20) UNIQUE,
    gyártó CHAR(20)
);
```

```
CREATE TABLE Felszolgál (
    teázó CHAR(20),
    tea VARCHAR(20),
    ár REAL,
    PRIMARY KEY (teázó, tea)
);
```

- egy relációban max egy PRIMARY KEY de bármennyi UNIQUE
  - PK nem lehet NULL, de unique igen

megszorítások

- olyan adatelemek közti kapcsolat amit fent kell tartani
- triggerek: megadott eseménykor bekövetkeznek
- lehetnek (idegen) kulcsok

```
FOREIGN KEY(tea) REFERENCES Teák(név));
```

- attribútum/érték alapú megszorítás: adott attribútum lehetséges értékeit korlátozza
- sor-alapú megszorítás: tuple-based constraint
- idegen kulcs megszorítások megőrzése
  - sérülhet ha kulcstáblában nem szereplő értékre hivatkozunk vagy törlünk kulcstáblából (lógó sorok hivatkozó táblában)
  - védekezés ellene: default vagy továbbgyűrűzés vagy set null
- idegen kulcs deklaráláskor megadhatjuk CASCADE vagy SET NULL stratégiát

```
FOREIGN KEY(tea)
    REFERENCES Teák(név)
    ON DELETE SET NULL
    ON UPDATE CASCADE
```

```
CREATE TABLE Felszolgál (
    teázó CHAR(20),
    tea CHAR(20) CHECK ( tea IN (SELECT név FROM Teák)),
    ár REAL CHECK ( ár <= 5.00 )
);
```

- attribútum alapú ellenőrzés csak beszúrásnál és módosításnál
- globális megszorítások: adatbázissémához tartoznak
  - bármely módosításnál ellenőriz

```
CREATE ASSERTION <név> CHECK (<feltétel>);
```

- triggerek: ECA szabályok (event-condition-action)

```
CREATE TRIGGER TeaTrig
BEFORE INSERT ON Felszolgál
REFERENCING NEW ROW AS ÚjSor
FOR EACH ROW
    WHEN (ÚjSor.tea NOT IN (SELECT név FROM Teák))
INSERT INTO Teák(név) VALUES(ÚjSor.tea);
```

előadás 04

- tranzakció: adatbázis lekérdezések, módosítások folyamata, végrehajtásuk tartós
- tranzakciófeldolgozó
  - konkurenciakezelő
  - naplózás és helyreállítás-kezelő
- hasznuk: több felhasználó egyidőben és el kell különíteni a folyamatokat
- ACID tranzakciók
  - Atomicity: összes vagy egyik sem hajtódik végre
  - Consistency: megszorítások megőrződnek
  - Isolation: úgy tűnik, hogy a folyamatok elkülönítve hajtódnak végre
  - Durability: befejeződött tranzakció módosításai nem vesznek el
- COMMIT: utasítás után tranzakció véglegesnek tekinthető
- ROLLBACK: tranzakció abortál

Elkülönítési szint	Piszkos adat olvasása	Nem ismételhető olvasás	Fantomadatok
READ UNCOMMITTED	Megengedett	Megengedett	Megengedett
READ COMMITTED	Nem Megengedett	Megengedett	Megengedett
REPEATABLE READ	Nem Megengedett	Nem Megengedett	Megengedett
SERIALIZABLE	Nem Megengedett	Nem Megengedett	Nem Megengedett

- elkülönítési szintek: milyen interakciók engedélyezettek az egyidőben végrehajtandó tranzakciók közt

```
SET TRANSACTION ISOLATION LEVEL X
-- SERIALIZABLE, REPEATABLE READ, READ COMMITTED, READ UNCO
MMITTED
```

- read uncommitted: nem véglegesített változtatásokat is olvas
- read committed: csak véglegesítettet de nem mindig ugyanazt
- repeatable read: adat újraolvasásánál másodszor is ugyanazt látjuk (hasonló read committedhez)

nézettáblák

```
CREATE [MATERIALIZED] VIEW <név> AS <lekérdezés>;
```

- nézettábla: reláció, def. tárolt táblákkal (alaptáblák) és más nézettáblákkal; kétféle:
- virtual: nem tárolódik db-ban; csak a relációt megadó lekérdezés
- materialized: kiszámítódik majd eltárolódik
- ugyanúgy kérdezhetőek le mint az adattáblák
- probléma: materialized táblák néha nem aktuálisak
  - megoldás: periodikus frissítésük

```
CREATE INDEX TeaInd ON Teák(gyártó);
CREATE INDEX EladásInd ON Felszolgál(teázó, tea);
```

- indexek: adatszerkezet amivel egy reláció sorait gyorsabban érhetjük el attr. értékei alapján
  - lehet hash tábla de ált. balanced search tree-vel (B-fák)
  - adott v értékre azokra a sorokra irányít ahol ez a v érték megjelenik
- hangolásnál: milyen indexet használjunk?
  - pro: gyorsabb keresés

    - kontra: módosításnál indexet is módosítani kell
- tuning advisors: lekérdezés terhelési kimutatás (query load)

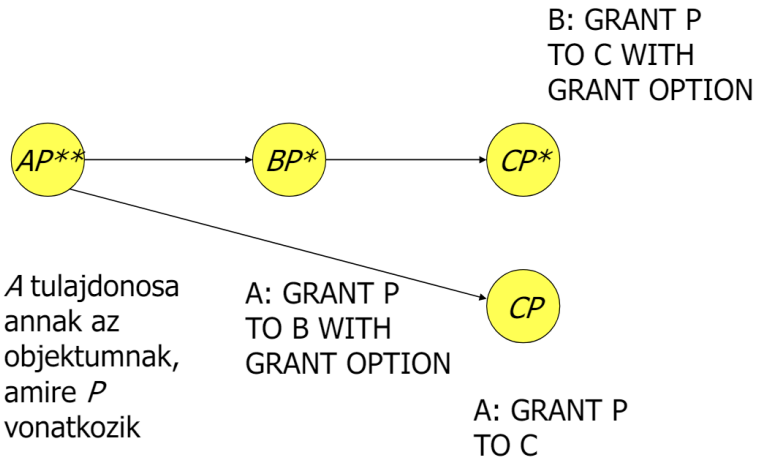
jogosultságok

```
GRANT <jogosultságok listája>
ON <reláció vagy másféle objektum>
TO <jogosultsági azonosítók listája>
[WITH GRANT OPTION];
```

- olvasás/írás/végrehajtás
- össz. 9 ~ sql-ben
- reláció lekérdezésének, sorok beszúrásának/törlésének/módosításának joga
- authorization ID alapján azonosítjuk a felhasználókat
    - külön jogosultsági azon. a PUBLIC
- with grant option: tovább is adhassa kapott ~at

```
REVOKE <jogosultságok listája>
ON <reláció vagy más objektum>
FROM <jogosultsági azonosítók listája>
```

- revoke opciók
    - cascade: azok a jogosultságok amiket az adott akitől éppen visszavonjuk a ~ot szintén törlődnek
    - restrict: nem hajtódik végre amíg visszavonandó ~tól függőek vannak, előtte azokat kell törölni



- engedélyezési diagramok pontjai: felhasználó/jogosultság/grant option?/tulajdonos-e
- X → Y él: X pontot használtuk Y jogosultságainak megadására
- AP pont: A jogosultsági azonosítójú felhasználó P joga
  - P*: with grant option
  - P**: P jog tulajdonosi viszonyból származik (következik belőle with grant option)
- ha A megadja a jogot B-nek, AP*[*]-ból húzunk egy élt BP[*]-be
- szabály: C felhasználónak van Q joga ha van él XP** pontból CQ/CQ*/CQ**-ba és P magában foglalja Q-t

előadás 05

relációs adatmodell-tervezés

- ha egy attribútum értékét ki tudjuk következtetni más attribútum(ok)ból → redundáns
- következtetési szabályok: **Armstrong axiómák**
- X → Y: R relációra megszorítás, **funkcionális függőségek**
  - ha két sor megegyezik X összes attribútumán, akkor Y attribútumain is meg kell, hogy egyezzenek
  - X, Y, Z attribútum-halmazok, A, B, C attribútumok
- jobboldalak szétvágása: X → A1A2...An akkor és csak akkor teljesül R-en, ha X → A1, X → A2, ..., X → An is teljesül

    - pl. A → BC ekvivalens A → B ÉS A → C kettőssel
- K **szuperkulcs** R-re, ha K funkcionálisan meghatározza R attribútumait
    - tul.képp K szuperkulcs ha a benne lévő attribútumok alapján meg lehet határozni a többi attr. értékét (pl. személyi szám alapján név, szül. dátum meghatározható)
- K **kulcs,** ha K szuperkulcs de egyetlen valódi részhalmaza sem szuperkulcs
    - pl. név + irányítószám nem szuperkulcs, mert a név vagy irányítószám nem hat. meg minden más adatot
- FF-ek kikövetkeztetése: inferring
- Armstrong-axiómák
    - Reflexivitás: $Y \subseteq X \subseteq R$ akkor X → Y (triviális függőségek)
    - Bővítés: ha X → Y teljesül, akkor tetszőleges $Z \subseteq R$-re XZ → YZ teljesül
    - Tranzitivitás: ha X → Y és Y → Z akkor X → Z
- R rel. és F FF-halmaz mellett: Y lezártja (closure): Y^+ összes olyan A halmaza, amire Y → A következik F-ből
    - kiszámítására egy algo.; kiindulás: $Y^+ = Y$
    - indukcióval: olyan FF-eket keresünk, melyeknek baloldala benne van Y^+-ban; ha X → A ilyen, akkor A-t hozzáadjuk
    - vége van ha már nem tudunk többet hozzáadni

előadás 06

- összes következmény FF megtalálása
    - motiváció: normalizálás → egy sémát több sémára bontunk fel
    - kiindulunk megadott FF-kből és megkeressük az összes nemtriviális FF-et
- exponenciális algo.
    - minden X-re kiszámítjuk lezártját
    - hozzáadjuk függőségekhez X → A-t minden A-ra $X^+ - X$-ből
    - dobjuk ki XY → A-t ha X → A is teljesül

- végül csak azokat az FFeket használjuk amikben csak a projektált attr.-ok szerepelnek

- *ABC*, *A* ->*B* és *B* ->*C* FF-kel. Projektáljunk *AC*-re.
  - $A^+ = ABC$; ebből *A* ->*B*, *A* ->*C*.
    - Nem kell kiszámítani AB^+ és AC^+ lezárásokat.
  - $B^+ = BC$; ebből *B* ->*C*.
  - $C^+ = C$; semmit nem ad.
  - $BC^+ = BC$; semmit nem ad.

- A kapott FF-ek: *A* ->*B*, *A* ->*C* és *B* ->*C*.
- *AC* -re projekció: *A* ->*C*.

TODO: ab tervezés C pdf 11. o

- relációssémák tervezése
  - cél: anomáliák megszüntetése
  - módosítási/törlési/beszúrási anomália

- *R* relációt, $\{A_1, A_2, \ldots, A_n\}$ attribútumokkal helyettesítsük $S(B_1, B_2, \ldots, B_m)$ és $T(C_1, C_2, \ldots, C_k)$ relációkkal úgy, hogy

1. $\{A_1, A_2, \ldots, A_n\} = \{B_1, B_2, \ldots, B_m\} \cup \{C_1, C_2, \ldots, C_k\}$
2. $S = \Pi_{B1, B2, \ldots, Bm}(R)$
3. $T = \Pi_{C1, C2, \ldots, Ck}(R)$

reláció felbontása

$$r = \Pi_{R1}(r) \mid X \mid \ldots \mid X \mid \Pi_{Rk}(r)$$

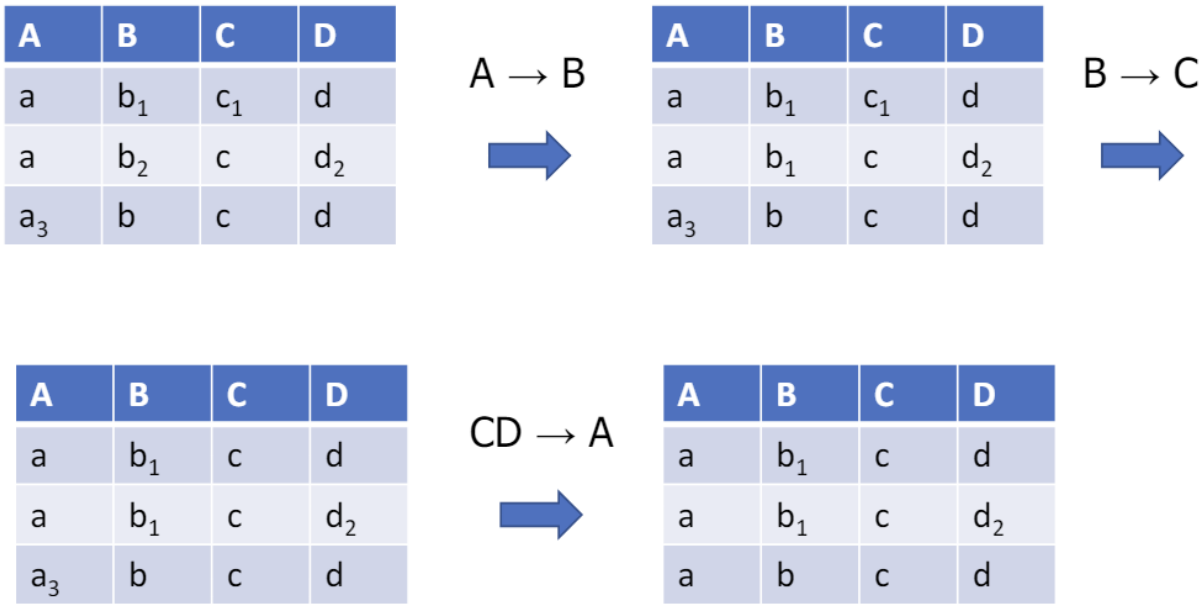
veszteségmentes felbontás feltétele

- veszteségmentes felbontás: itt r egy R sémájú reláció, R attr.-ainak részhalmazai R1, ... Rk

BCNF

- Boyce-Codd normálforma: BCNF forma

- ha minden X → Y nemtriviális FF-re R-ben X szuperkulcs
- nemtriviális: Y nem része X-nek
- szuperkulcs: tartalmaz kulcsot (ő maga is lehet kulcs)
- BCNF-re felbontás: R rel.-va és F FF-ekkel
  - X lezártját kiszámoljuk, ha nem szerepel benne az összes attr. akkor nem szuperkulcs
- veszteségmentes felbontás de exponenciális lehet :((



Chase-teszt

- Chase-teszt veszteségmentességhez
  - F-beli függőségekkel egyenlővé tesszük azokat aminek ua.-nak kell lennie hogy a függőség ne sérüljön
  - szimbólumok minden előfordulását helyettesíteni kell az új értékkel
  - tablót kapunk → ha t szerepel benne veszteségmentes

TODO: ab terv E

többértékű függőségek

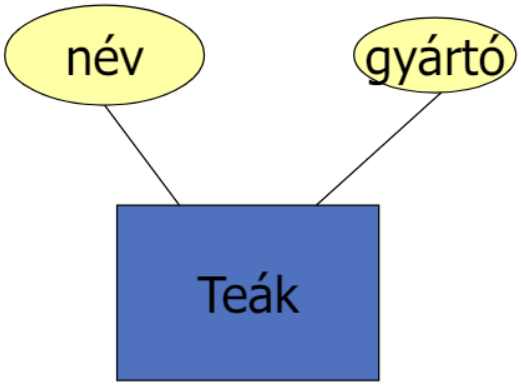
Funkcionális függőségek	Többértékű függőségek	Vegyes függőségek
A1 (reflexivitás): $Y \subseteq X$ esetén $X \to Y$.	**A4 (komplementer):** $X \to\to Y$ és $Z=R-XY$ esetén $X \to\to Z$.	**A7 (funkcionálisból többértékű):** $X \to Y$ esetén $X \to\to Y$.
A2 (bővíthetőség): $X \to Y$ és tetszőleges Z esetén $XZ \to YZ$.	**A5 (bővíthetőség):** $X \to\to Y$ és tetszőleges $V \subseteq W$ esetén $XW \to\to YV$.	**A8 (többértékűből és funcionálisból funkcionális):** $X \to\to Y$ és $W \to S$, ahol $S \subseteq Y$, $W \cap Y = \varnothing$ esetén $X \to S$.
A3 (tranzitivitás): $X \to Y$ és $Y \to Z$ esetén $X \to Z$.	**A6 (tranzivitás):** $X \to\to Y$ és $Y \to\to S$ esetén $X \to\to S-Y$.	

- multivalued dependency
- TÉF: R felett X →→ Y teljesül
- bármely két sor ahol X megegyezik, ott Y felcserélhető (keletkező két új sor R-beli lesz)
- pl. vendégek telefonszámai függetlenek általuk kedvelt teáktól azaz név →→ tel és név →→ kedveltTeák
- **független az FF-ektől!!** de minden FF TÉF
- komplementálás: ha X →→ Y és Z minden más attr. halmaza akkor X →→Z
- sem jobb- sem baloldalakat nem bonthatjuk fel
- TÉF-ek okozta redundanciát BCNF nem szünteti meg → 4. normálforma 😐
  - úgy kezeljük a TÉF-eket mint az FF-eket BCNF-ben
- triviális TÉF-ek: legyen R, X és Y részhalmazai R attr.-ainak
  - X →→Y automatikusan fennáll ha $Y \subseteq X$
  - X →→ Y aut. fennáll ha $X \cup Y = R$
- R 4. normálformában van ha minden X →→ Y nemtriv. TÉF esetén X szuperkulcs
  - nemtriviális: Y nem részhalmaza X-nek, X és Y együtt nem adják ki az összes attribútumot

- ha R 4NF-ben van akkor BCNF-ben is

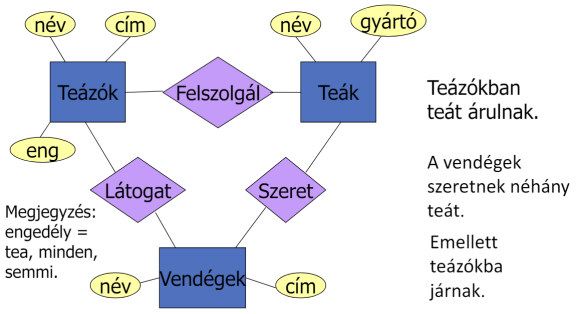
előadás 07

- egyed-kapcsolat modell (entity-relationship model): adatbázissémát vázolhatjuk fel vele
  - bizonyos megszorításokat is, de műveleteket nem ábrázolhatunk vele
- tervek: egyed-kapcsolat diagram
- később diagramot relációs adatbázissémává alakítjuk
- egyed/entitás
- egyedhalmaz/entity set: OOP-beli osztályhoz hasonló
  - egy téglalap a diagramon
  - értéke a hozzá tartozó egyedek halmaza
- attribute/attribútum: atomi értékűek
  - megfelelő egyedhalmazzal összekötött ovális 🥔



Teák egyedhalmaznak két attribútuma van, minden Teák-béli entitás értéket kap ezeken

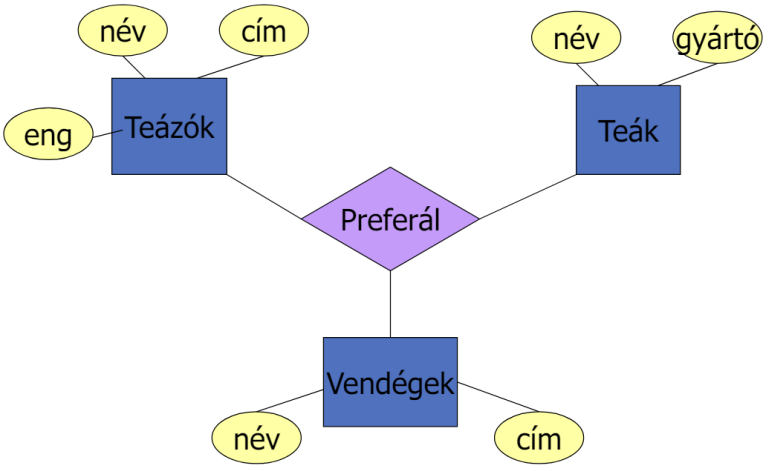
- kapcsolat/relationship: kettő vagy több egyedhalmazt köt össze
  - értéke a kapcsolathalmaz (relationship set), sorok egy halmaza amiknek minden eleme egy a kapcsolatban résztvevő egyedhalmazból való
  - rombusszal jelöljük, a résztvevő egyedhalmazok téglalapjaival kötjük össze



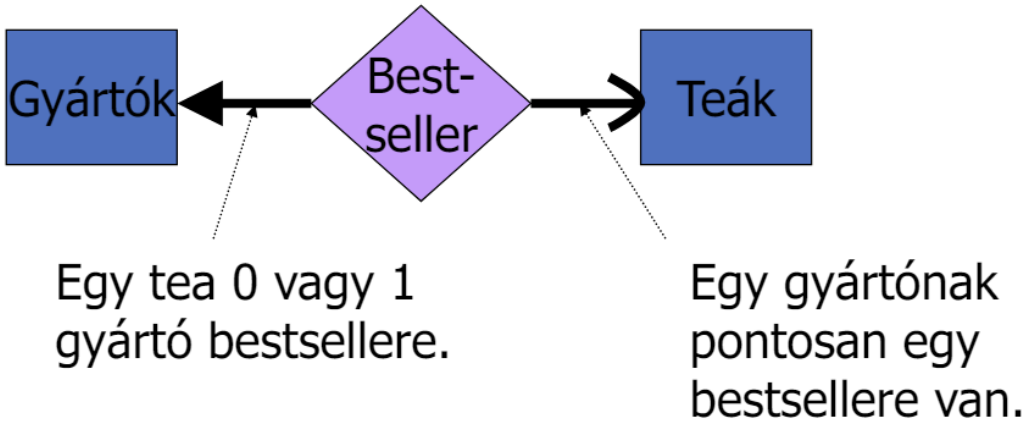
Teázó	Tea
Green Teahouse	Brisk
Green Teahouse	Earl Grey
Frank's Twinings	Brisk
Frank's Twinings	Pyramid
Frank's Twinings	Earl Grey

Felszolgál kapcsolathoz tartozó kapcsolathalmaz

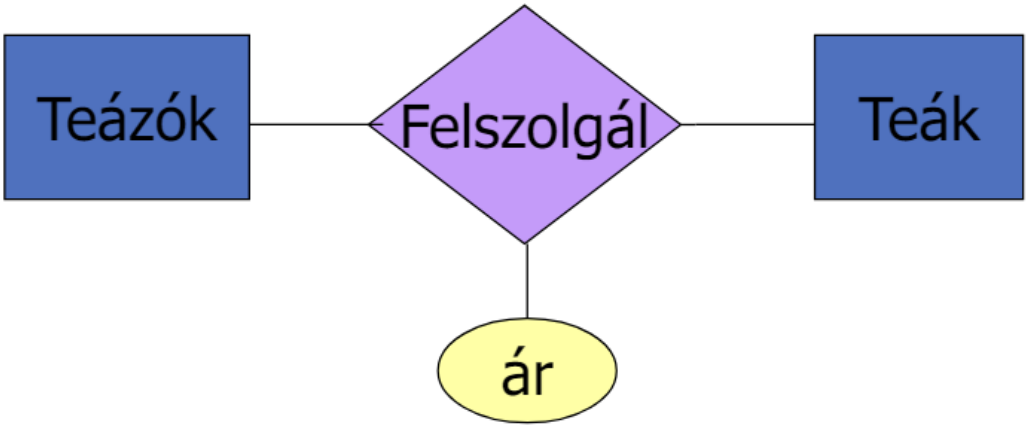
- többirányú kapcsolatok: több mint 2 egyedhalmazt kötnek össze
  - pl. egyes vendégek egyes teát csak biz. teázókban fogyasztanak



- sok-sok kapcsolat: minden résztvevő egyedhalmaz egyedei több más egyedhalmazhoz kapcsolódhatnak
- sok-egy kapcsolat: 1. egyedhalmaz elemei max. 1-hez, másik halmaz akármennyi 1. egyedhalmaz-beli egyedhez kapcsolódhatnak
  - pl. Vendégek-Teák Kedvenc kapcsolata (egy embernek max. egy kedvence, egy tea bármennyi embernek kedvence lehet)
- egy-egy kapcsolat: minden entitás max. egy másikhoz kapcsolódhat

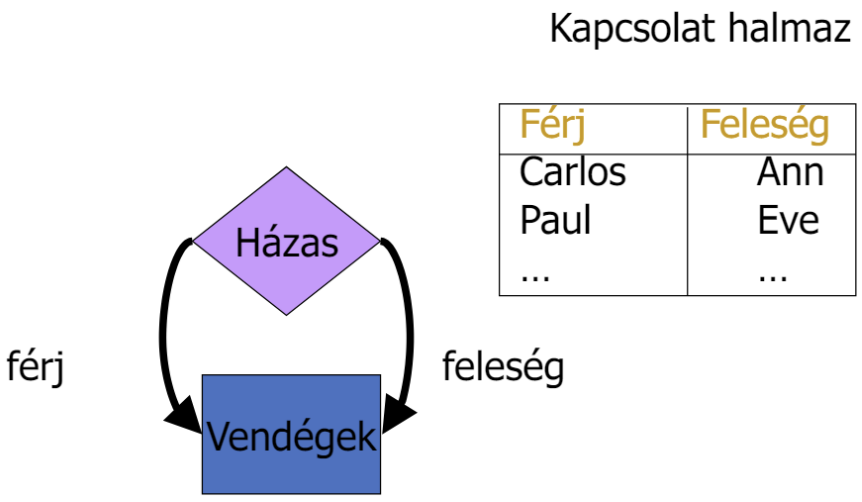


- számosság/multiplicitás/multiplicity
  - sok-egy kapcsolat “egy oldalát” nyíl jelzi
  - egy-egy kapcsolatnak mindkét végén nyíl van
  - lekerekített nyíl: pontosan egy

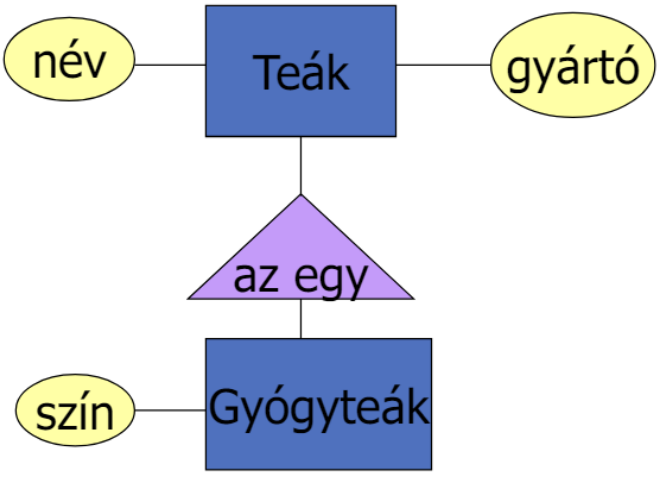


kapcsolat egy attribútuma

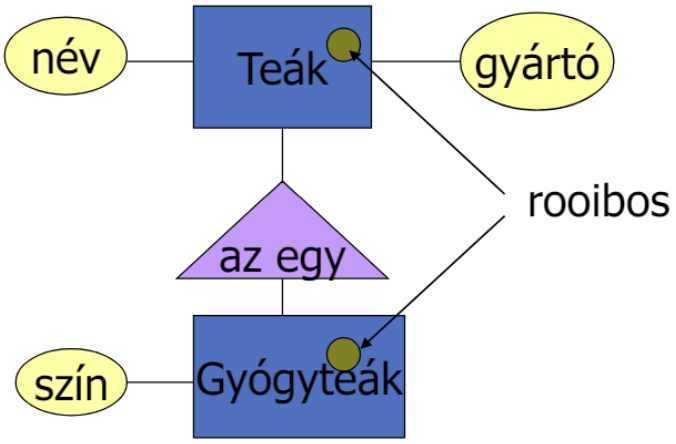
- szerep/roles: néha egy egyedhalmaz többször is megjelenik egy kapcsolatban
  - megfelelő éleket címkézzük és a címkéket



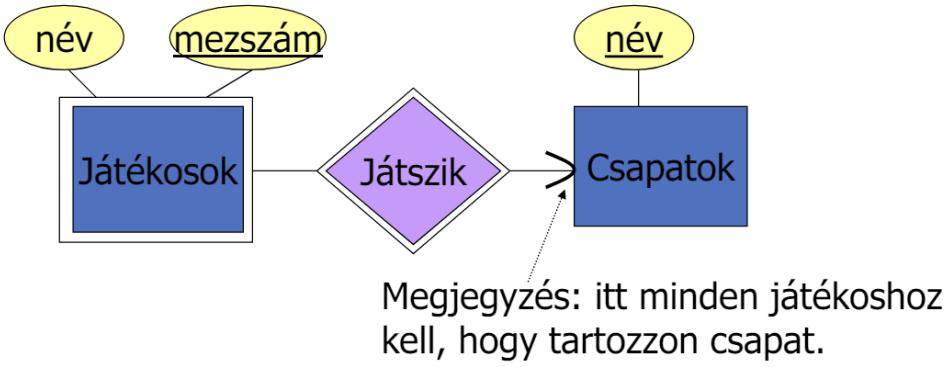
- alosztályok/subclasses: speciális eset; kevesebb egyed = több tulajdonság
  - pl. gyógyteák mind teák is



- t.f.h. alosztályok rendszere fát alkot
  - kapcsolatokat az ‘az-egy’ háromszögek jelölik (superclassra mutat a csúcs)
- OOP-ben minden obj. csak egy osztálynak lehet eleme
  - E/K-ben nem → egyedeknek minden alosztályban vannak reprezentánsai
  - ha e egyed szerepel egy alosztályban, akkor szerepel annak ősosztályaiban is



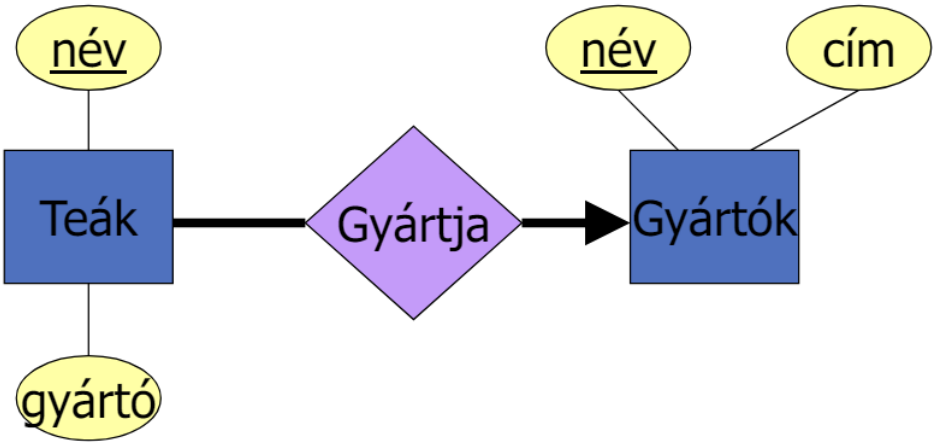
- kulcs: nem lehet 2 egyed azonos kulccsal; minden egyedhalmazhoz meg kell adni egy kulcsot
- gyenge egyedhalmazok/weak entity set: E egyedhalmaz gyenge ha a belőle induló sok-egy kapcsolatot követve a kapcsolódó egyedek kulcsértékeire is szükségünk van annak meghatározásához
  - pl. név gyenge adathalmaz, mert előfordulhat hogy két embernek ua. a neve
  - jelölése: dupla téglalap
  - támogató/supportive egyedhalmazba mutató rombusz dupla szintén



- minden egyednek a gyenge egyedhalmazból pontosan egy egyedhez kell kapcsolódni a támogató egyedhalmazból
- gyenge e.halmaz kulcsa a saját aláhúzott és támogató e.halmazok kulcsaiból áll

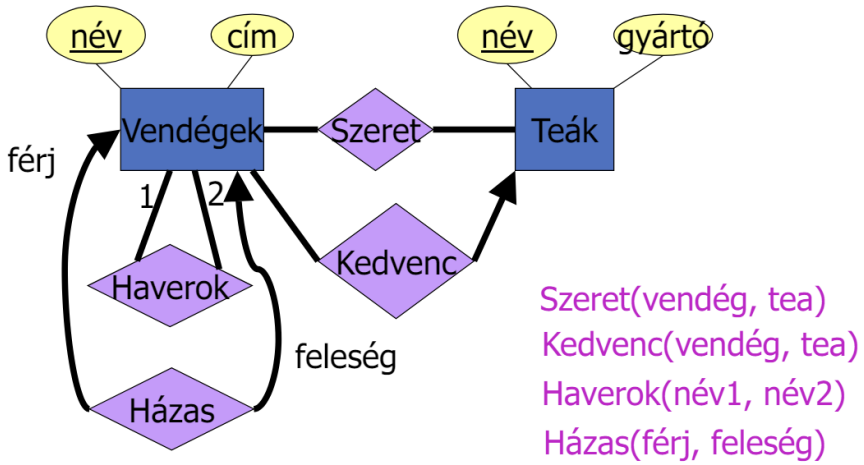
előadás 08

- tervezési technikák
  - redundancia kerülése
  - gyenge e.halmazok óvatos használata
  - ne használjunk e.halmazt ha egy attr. is elég lenne



redundáns: a gyártó neve kétszer is szerepel

- redundanciakerülés
  - redundancia: ua. dolgot többféleképp is feltüntetjük
  - növeli az inkonzisztencia veszélyét (módosítási anomália)
- egyedhalmaz vs. attribútum
  - egyedhalmaz: legalább egy nem kulcs attr.-ának lennie kell
  - vagy a ‘sok’ végén szerepel egy egy-sok kapcsolatnak



- diagramból reláció
  - egyedhalmazból reláció
  - kapcsolatból reláció, attribútumai csak az összekapcsoltak kulcsattr.-ai és a kapcsolat attr.-ai
  - össze lehet vonni relációkat
    - de a sok-sok kapcsolat nem összevonható!

előadás 09

- O-O modell
  - több más adattípust is támogat
- objektum-relációs adatmodell: O-O és relációs modell együtt?
- relációk → objektumrelációk
- attribútumokhoz szerkesztett strukturált típusok, metódusok, sorazonosítók (row IDs), hivatkozások (references)
- beágyazott relációk: reláció attr. típusa nem csak atomi típus lehet hanem pl. egy másik reláció is
  - pl. Rel1(Attr1, Attr2(Attr2A, Attr2B), Attr3)
- lehet hivatkozni adott sémájú sor(ok)ra
  - ha A attr. típusa egy R relációsémájú sorhivatkozás: A(*R)
  - ha hivatkozási halmaz: A({*R})
- felhasználó által definiált adattípus (UDT/user defined data type): O-O osztály def.-ja
  - adatszerkezet & metódusai
  - használati mód: sortípus (row type) → relációt mint adattípus kezeljük
  - vagy mint egy reláció attribútumának típusa

```
CREATE TYPE <típusnév> AS OBJECT (<attribútum-típus párok l
istája>);
CREATE TYPE TeázóTípus AS (név CHAR(20), cím CHAR(20));
```

- ha T egy UDT → ref T egy T-re történő hivatkozás típusa (= pointer egy T típusú egyedre)
    - O-O rendszerekben objektumazonosítóknak (object ID, OID)
    - OID élete végéig azonosít egy objektumot
    - REF látható (OID nem)

```
CREATE TABLE <táblanév> OF <típusnév>;
CREATE TABLE Teázók OF TeázóTípus;
```

- mindegyik UDT-nek van egy típus konstruktora
    - illetve generátorai (getter basically): A()
    - mutátorai (setter): A(v) ahol v az érték
    - ezen metódusok nevei megegyeznek az attr.-éval

```
SET újTeázó = TeázóTípus();
    újTeázó.név('Joe''s Teahouse');
    újTeázó.cím('Maple St.');
INSERT INTO Teázók VALUES(újTeázó);
```

- UDT lehet egy attribútum típusa is

```
SELECT dd.kedvencTea.név
FROM Vendégek dd;
```

- Ref/hivatkozás követése: A → B csak akkor értelmes, ha A REF T típusú és T típus objektum mezője/komponense B
    - A → B az A által hivatkozott obj. B mezőjét jelöli
- DEREF: értékét állítja elő pl. TeaTípus('Brisk', 'Lipton')

```
CREATE TYPE FelszolgálTípus AS OBJECT (
    teázó REF TeázóTípus,
    tea REF TeaTípus,
    ár FLOAT,
MEMBER FUNCTION árYenben(ráta IN FLOAT) RETURN FLOAT,
```

```
PRAGMA RESTRICT_REFERENCES(árYenben, WNDS)
);
```

- metódusdefiníció: CREATE TYPE-ban vagy CREATE TYPE BODY-ban (PL/SQL szintaxissal :)))))
  - SELF változó: objektumra hivatkozik (this)
  - WNDS: nem változtatja db állapotát

```
SELECT ff.tea.név,
ff.árYenben(114.0)
FROM Felszolgál ff
WHERE ff.teázó.név = 'Joe''s Teahouse';
```

- rendező metódusok (order methods): mindegyik T UDT két metódust definiálhat (SQL-99)
  - EQUAL és LESSTHAN
  - mindegyik metódus T típusú objektumot kap és egy másik T típusúra alkalmazza
- Oracle-ben: bármelyik metódust rendező metódusnak definiálhatjuk
  - visszatérő értékük <0, =0, >0 lehet SELF objektumhoz viszonyítva

```
CREATE TYPE BODY TeázóTípus AS
ORDER MEMBER FUNCTION
before(teázó2 TeázóTípus) RETURN INT IS
BEGIN
IF SELF.név < teázó2.név THEN RETURN –1;
ELSIF SELF.név = teázó2.név THEN RETURN 0;
ELSE RETURN 1;
END IF;
END;
END;
/
```

- Oracle beágyazott táblák: megengedi, hogy a sorok egyes komponensei teljes relációk legyenek

- ha T egy UDT, létrehozhatunk S reláció típust belőlük

```
CREATE TYPE S AS TABLE OF T ;

CREATE TYPE TeaTípus AS OBJECT (
    név CHAR(20),
    fajta CHAR(10),
    szín CHAR(10)
);

CREATE TYPE TeaTáblaTípus AS TABLE OF TeaTípus;

CREATE TABLE Gyártók (
    név CHAR(30),
    cím CHAR(50),
    teák TeaTáblaTípus
)
NESTED TABLE teák STORE AS TeaTábla;
```

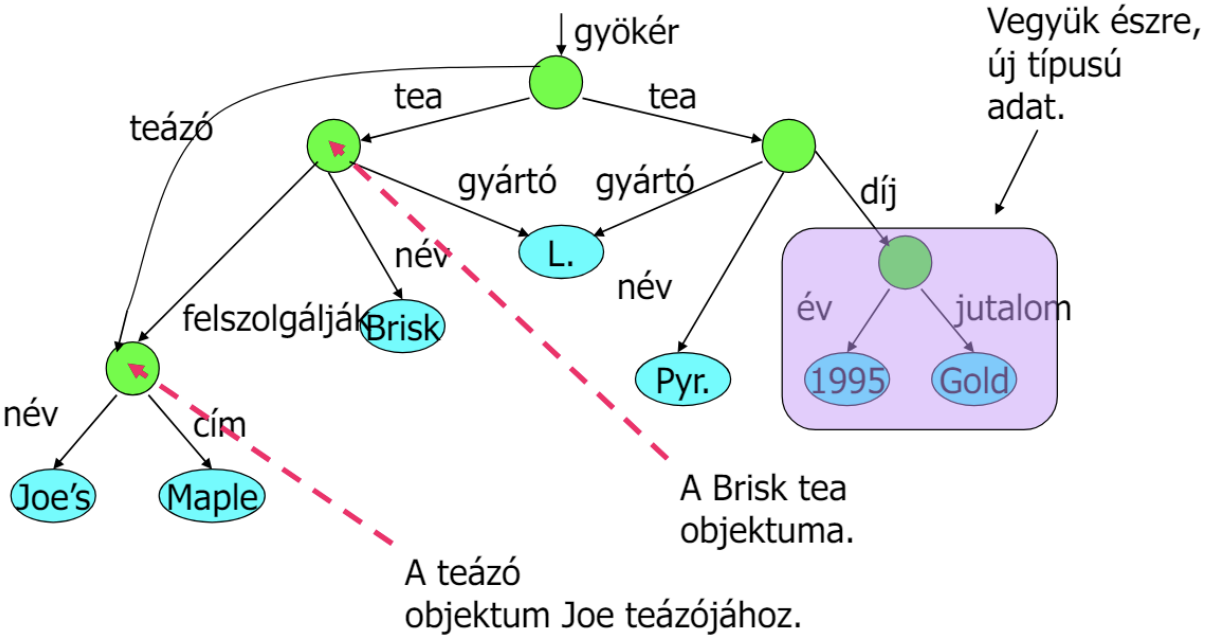
- beágyazott táblát hagyományos táblává alakítani: TABLE() utasítással

```
CREATE TABLE aruhazak (
    aruhaz_id NUMBER,
    nap DATE,
    vevok vevo_tab
)
NESTED TABLE vevok STORE AS aruhaz_vevok
( NESTED TABLE cimek STORE AS aruhaz_cimek );
```

előadás 10

- féligstrukturált adat: fákon alapuló adattípus
  - adatok rugalmas kezeléséért
  - motiváció: dokumentumok megosztása
- pontok = objektumok
- élek címkéi tul.képp attribútumok

- atomi értékek: levélcsúcsokban (leaf nodes) tárolódnak
- példa: adatgráf



- XML = Extensible Markup Language
  - jelentés, szemantika leírásához tageket használ
  - dokumentum deklarációval kezdődik, amit egy <?xml ... ?> tag határol

```
<?xml version = “1.0” encoding = “utf-8” ?>
<?xml version = ”1.0” standalone = ”yes” ?>
```

- tagek case sensitive-ek!
-

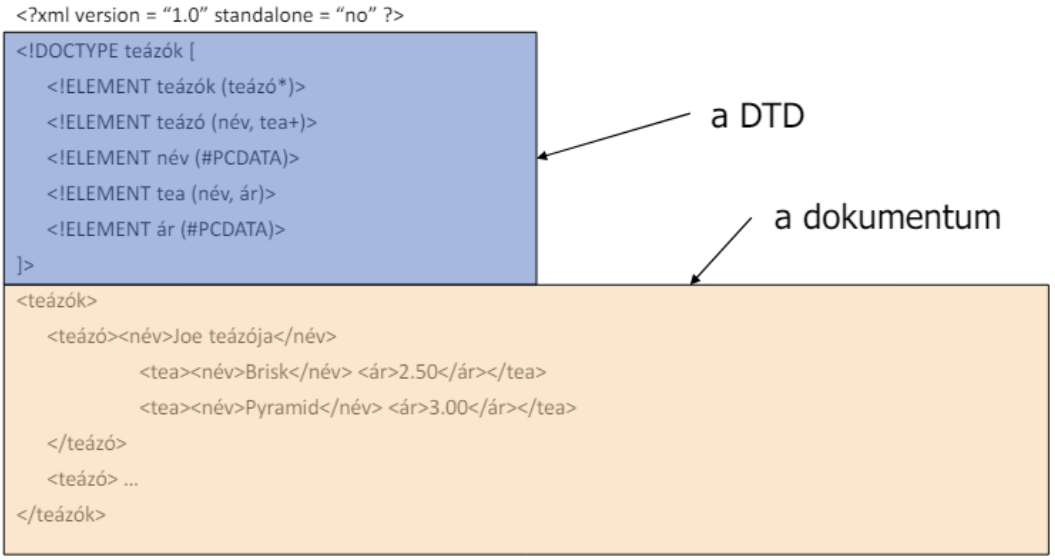
```
<?xml version = “1.0” stan
dalone = “yes” ?>
<teázók>
    <teázó>
        <név>Joe teázója</
név>
        <tea><név>Brisk</n
év>
        <ár>2.50</ár></tea
```

```
<!DOCTYPE teázók [
    <!ELEMENT teázók (teáz
ó*)>
    <!ELEMENT teázó (név,
tea+)>
    <!ELEMENT név (#PCDAT
A)>
    <!ELEMENT tea (név, á
r)>
```

```
>
        <tea><név>Pyramid
</név>
        <ár>3.00</ár></tea
>
    </teázó>
    <teázó> …
</teázók>
```

```
    <!ELEMENT ár (#PCDATA)
>
]>
```

- jól formált/well formed XML megengedi, hogy saját tageket vezessünk be
  - nem hiányzik a deklaráció
  - minden tag le van zárva
  - egymásba ágyazhatóak a tagek de nem fedhetnek át
  - egy gyökér jelölő (root tag) van
- érvényes/valid XML illeszkedik egy megadott sémára (DTD/XML séma)



- DTD
  - * = 0 vagy több, ... szokásos jelölések
  - |: vagy
  - alelemek felsorolás sorrendjében kell hogy kövessék egymást

- #IMPLIED: opcionális (van #REQUIRED)
- ID és IDREF: egymásra mutatnak; XML ezért inkább gráf mint fa
- üres elem/empty element: lezárjuk nyitó taget />

```
<xs:complexType name = ”teaTípus”>
    <xs:sequence>
        <xs:element name = ”név”
            type = ”xs:string”
            minOccurs = ”1” maxOccurs = ”1” />
        <xs:element name = ”ár”
            type = ”xs:float”
            minOccurs = ”0” maxOccurs = ”1” />
        </xs:sequence>
</xs:complexType>
```

- XML sémák: szintén XML dokumentumokat lehet leírni, de több megszorítást lehet leírni mint DTD-vel
    - ez maga is egy XML dokumentum
- xs:simpleType-vel felsorolásokat adhatunk meg & az alaptípusra vonatkozó megszorításokat; name & xs:restriction attribútumai lehetnek
- xs:restriction
    - base attribútum: melyik egyszerű típus pl xs:integer
    - {min, max}{Inclusive, Exclusive} alsó-felső korlátok

```
<xs:simpleType name = ”engedély”>
    <xs:restriction base = ”xs:string”>
        <xs:enumeration value = ”minden” />
        <xs:enumeration value = ”csak tea” />
        <xs:enumeration value = ”csak üdítő” />
    </xs:restriction>
</xs:simpleType>
```

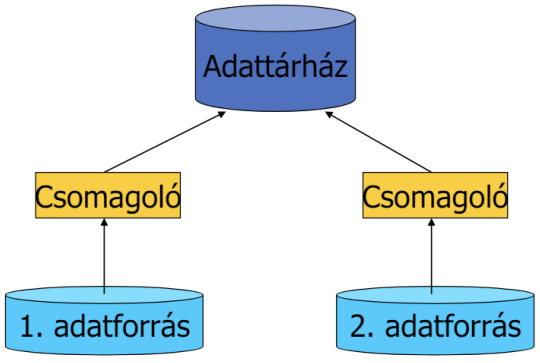
```
<xs:simpleType name = ”ár”>
    <xs:restriction
        base = ”xs:float”
```

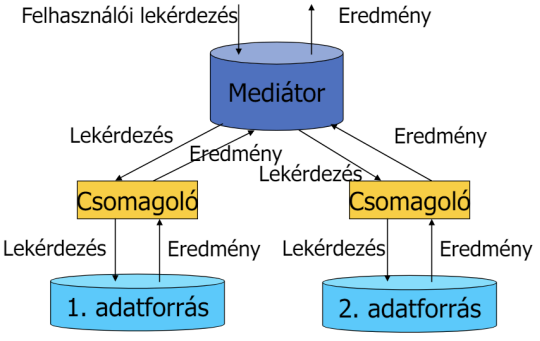
```
        minInclusive = ”1.00”
        maxExclusive = ”5.00” />
</xs:simpleType>
```

- xs:elementhez tartozhat xs:key
  - ezen elemen belül minden adott szelektor ösvényen elérhető elemnek egyedi értékkel kell rendelkeznie ezen fielden/mezőn

```
<xs:element name = ”teázók” … >
. . .
    <xs:key name = ”teázóKulcs”>
        <xs:selector xpath = ”teázó”/>
        <xs:field xpath = ”@név” />
    </xs:key>
. . .
</xs:element>
```

- idegen kulcs: xs:keyref
- információintegrálás: több helyről érkező adatot hogyan integrálunk? 2 approach:





előadás 11

-